

CS307 Spring 2026 — Database Project Part 1 Report

Basic Information of the Group

Field	Member 1	Member 2
Name	马奕铭	李明龙
Student ID	12410614	12411830
Lab Session	Mon7,8	Mon7,8

Contribution Breakdown

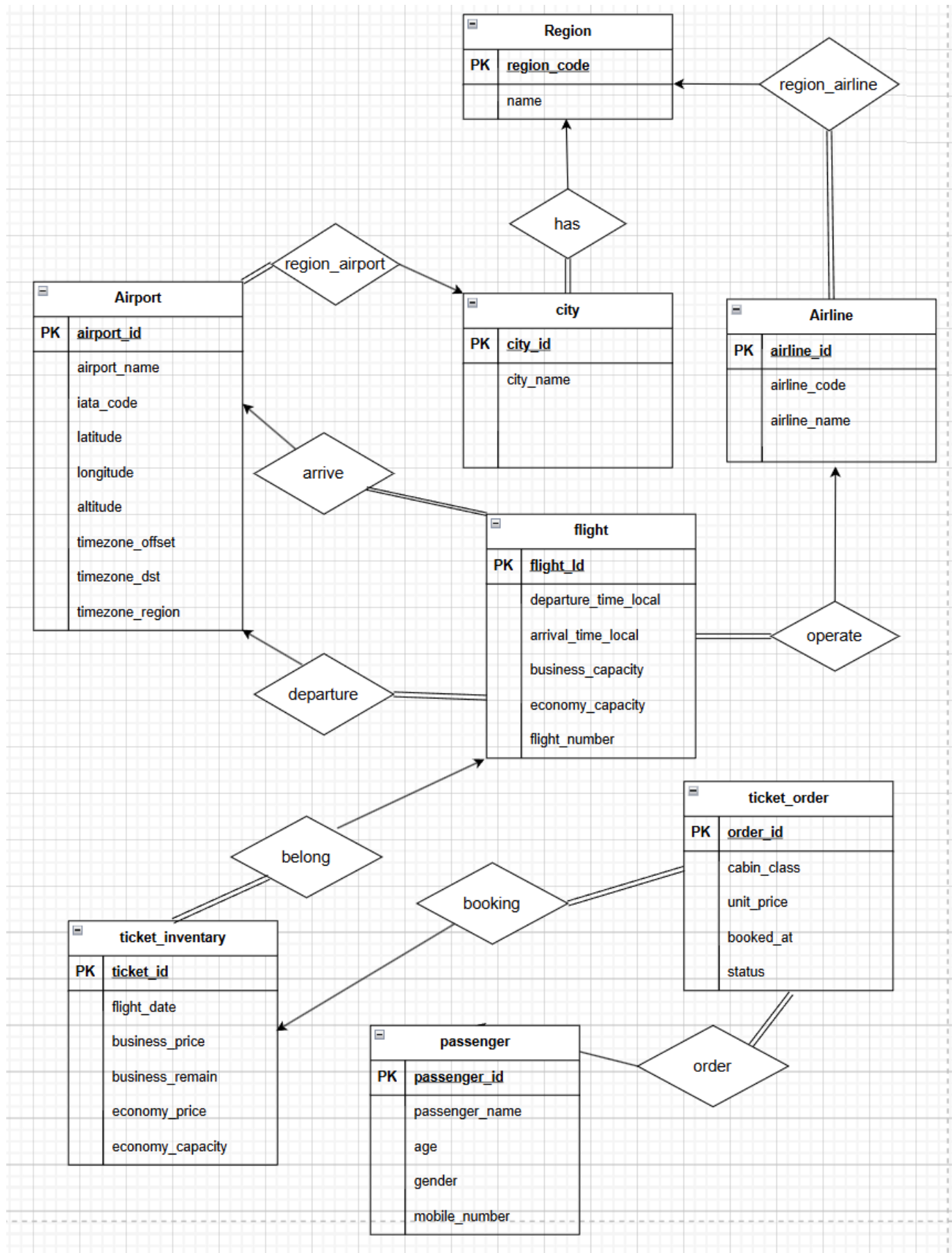
Task / Part	Member 1	Member 2
Task 1 – E-R Diagram (er_diagram.mmd)	-	✓
Task 2 – Database Design (schema.sql)	✓	-
Task 3.1 – Import scripts (import_data.py)	✓	-
Task 3.2 – Accuracy SQL (task3_accuracy_queries.sql)	✓	-
Task 4 – CRUD CLI (cli.py)	-	✓

Task 1: E-R Diagram (20 %)

Diagramming tool

draw.io

E-R Diagram



Entity and Relationship Overview

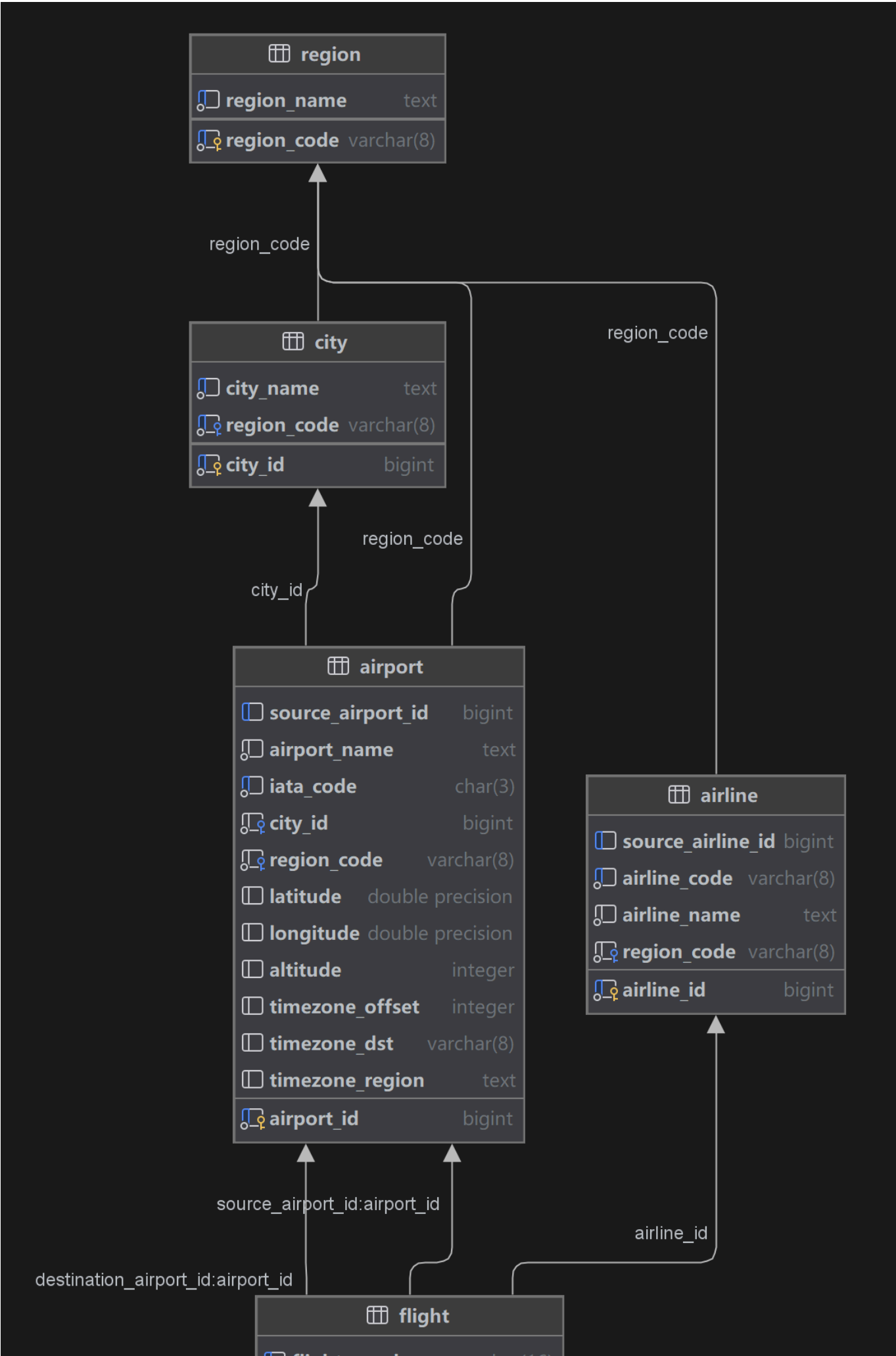
Entity	Key Attributes	Role
REGION	<code>region_code</code> (PK), <code>region_name</code>	Geographic region for cities and airlines
CITY	<code>city_id</code> (PK), <code>city_name</code> , <code>region_code</code> (FK)	City within a region
AIRPORT	<code>airport_id</code> (PK), <code>iata_code</code> (UK), <code>city_id</code> (FK)	Airport in a city
AIRLINE	<code>airline_id</code> (PK), <code>airline_code</code> (UK), <code>region_code</code> (FK)	Airline operating flights
PASSENGER	<code>passenger_id</code> (PK), <code>mobile_number</code> (UK)	Registered passenger
FLIGHT	<code>flight_id</code> (PK), <code>flight_number</code> (UK), <code>airline_id</code> (FK), source/destination airport FKs	A recurring flight route
TICKET_INVENTORY	<code>ticket_id</code> (PK), <code>flight_id</code> (FK), <code>flight_date</code> (UK per flight)	Daily ticket slot for a flight
TICKET_ORDER	<code>order_id</code> (PK), <code>passenger_id</code> (FK), <code>ticket_id</code> (FK)	A passenger's booking record

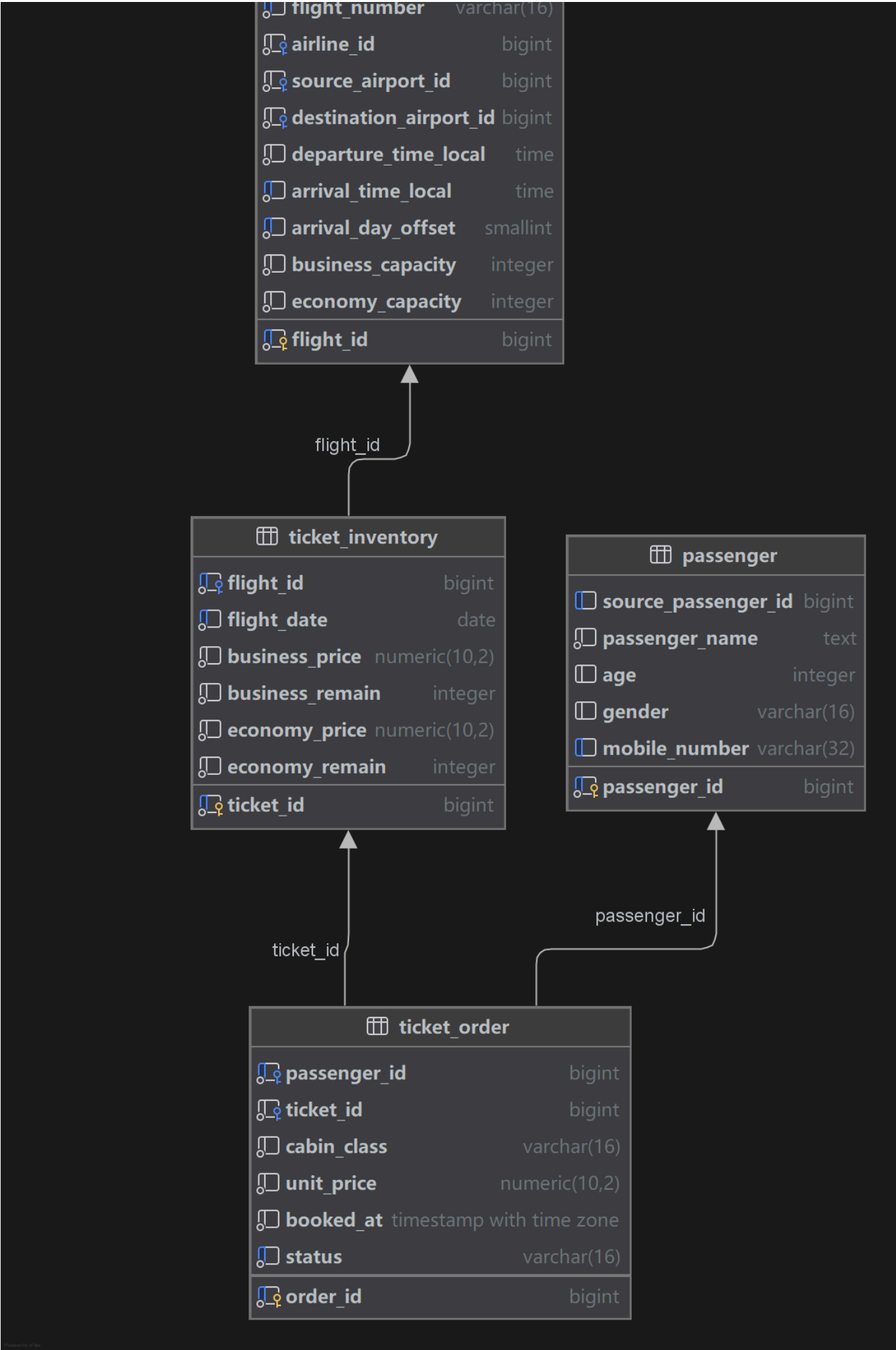
Key relationships:

- REGION **contains** many CITIES; each CITY belongs to one REGION.
- CITY **has** many AIRPORTs; each AIRPORT belongs to one CITY.
- REGION **has** many AIRLINEs; each AIRLINE is registered in one REGION.
- AIRLINE **operates** many FLIGHTs; each FLIGHT is operated by one AIRLINE.
- AIRPORT participates in FLIGHT as both **departure** and **arrival** endpoint.
- FLIGHT **schedules** many TICKET_INVENTORYs (one per date).
- PASSENGER **places** many TICKET_ORDERs; each TICKET_ORDER references one TICKET_INVENTORY.

Task 2: Database Design (20 %)

DataGrip E-R Diagram





Brief Design Description

This database is designed for flight search, ticket inventory management, booking, and cancellation workflows.

Reference entities include Region, City, Airport, Airline, and Passenger.

Operational entities include Flight, Ticket Inventory, and Ticket Order.

The complete DDL definitions are submitted separately in schema.sql.

Table and Column Meanings

Table	Meaning of the table	Main columns and meanings
region	Stores geographic regions used by cities and airlines	region_code: region identifier (primary key); region_name: full region name
city	Stores cities, each linked to one region	city_id: (primary key); city_name: city name; region_code: foreign key to region
airport	Stores airport master data	airport_id: (primary key); source_airport_id: source CSV id; airport_name: airport full name; iata_code: unique airport code; city_id: foreign key to city; latitude/longitude/altitude/timezone fields: location and timezone attributes
airline	Stores airline master data	airline_id: (primary key); source_airline_id: source CSV id; airline_code: unique airline code; airline_name: airline full name; region_code: foreign key to region
passenger	Stores passenger profile data	passenger_id: (primary key); source_passenger_id: source CSV id; passenger_name: full name; age/gender/mobile_number: personal attributes, mobile_number used as unique contact key
flight	Stores recurring flight routes and schedule templates	flight_id: (primary key); flight_number: unique flight number; airline_id: foreign key to airline; source_airport_id/destination_airport_id: route endpoints; departure_time_local/arrival_time_local: scheduled times; arrival_day_offset: same-day or next-day arrival; business_capacity/economy_capacity: seat capacities
ticket_inventory	Stores date-specific inventory and price for each flight	ticket_id: (primary key); flight_id: foreign key to flight; flight_date: operation date; business_price/economy_price: cabin prices; business_remain/economy_remain: remaining seats
ticket_order	Stores booking and cancellation records	order_id: (primary key); passenger_id: foreign key to passenger; ticket_id: foreign key to ticket_inventory; cabin_class: booked cabin;

Table	Meaning of the table	Main columns and meanings
		unit_price: transaction price; booked_at: booking time; status: booked or cancelled

Design Notes

- 1. The model separates recurring schedule data (flight) from date-specific inventory data (ticket_inventory), reducing redundancy.
- 2. City uniqueness is handled by city_name plus region_code, which avoids ambiguity for same-name cities in different regions.
- 3. arrival_day_offset is stored as an atomic integer field to represent next-day arrivals and support accurate time filtering.
- 4. Foreign keys enforce referential integrity across all major relationships.
- 5. Full constraints and indexes are defined in schema.sql (submitted as a separate file).

Task 3: Data Import (30 %)

Task 3.1 — Script Description (5 %)

Scripts submitted

Script name	Description
import_data.py	Single Python script that reads all five CSV files from the Archive/ directory and imports them into PostgreSQL in the correct dependency order. Also applies schema.sql before importing.
schema.sql	DDL file executed by import_data.py to create tables and indexes before any data is inserted.

How to run the import

Prerequisites

- 1. PostgreSQL running (tested on version 14+).
- 2. Python 3.9+ with the psycopg2 package installed (pip install -r requirements.txt).
- 3. CSV files present in Archive/ (airline.csv, airport.csv, passenger.csv, region.csv, tickets.csv).

Steps

```
Step 1 – Create the target database (run once):
cd C:\PostgreSQL\bin
.\psql -U postgres -c "CREATE DATABASE db_project_1;"

Step 2 – Run the import script:
cd "C:\Users\Yiming
Ma\Desktop\Claude_code_projects\DB_project_1\DB_project_1_by_copilot"
```

```
python import_data.py --host localhost --port 5432 --user postgres --password
123456 --database db_project_1
```

Step 3 – Verify output:

The script prints row counts for every table after import, for example:

Import completed. Row counts:

```
region:      XX
city:        XX
airport:     XX
airline:     XX
passenger:   XX
flight:      XX
ticket_inventory: XX
```

Import logic

The script processes tables in dependency order to satisfy all foreign-key constraints:

1. **Region codes** (`build_region_codes`) — Merges codes from `region.csv` with regions appearing in the other CSVs. Duplicate/alias region names are normalised (e.g., "Hong Kong SAR of China" → "Hong Kong").
2. **Regions** (`upsert_region`) — Bulk-insert with `ON CONFLICT UPDATE`.
3. **Cities** (`upsert_cities`) — Derived from `airport.csv` and `tickets.csv`; inserted with `ON CONFLICT DO NOTHING` on the `(city_name, region_code)` unique constraint.
4. **Airports** (`upsert_airports`) — Inserted from `airport.csv`; airports referenced in tickets but missing from the CSV are synthesised with the "{city} {iata} Airport" name pattern.
5. **Airlines** (`upsert_airlines`) — From `airline.csv`, matched to region codes.
6. **Passengers** (`upsert_passengers`) — From `passenger.csv`.
7. **Flights & ticket inventory** (`upsert_flights_and_tickets`) — Flight rows are deduplicated by flight number; `arrival_day_offset` is parsed from the (+1) suffix in arrival time strings. Ticket inventory rows are deduplicated on `(flight_id, flight_date)`.

Task 3.2 — Data Accuracy Checking (15 %)

Query 1 —Example: `:region_code = 'CN'`

Query 2 —Example: `:city_name = 'Taipei'`

Query 3 —Example: `:region_code = 'TW'`

Query 4 —Example: `:source_iata_code = 'FCO', :destination_iata_code = 'LTN'`

Query 5 —Example: `:flight_date = '2026-02-01', :departure_city = 'Rome', :arrival_city = 'London'`

Query 6 —Example: `:departure_time_after = '08:00:00', :arrival_time_before = '23:00:00'`

Note on the "arrive before 11:00" case: because arrival times that cross midnight are stored with `arrival_day_offset = 1`, adding the condition `arrival_day_offset = 0` ensures next-day arrivals are excluded before applying the time comparison. This is both correct and efficient because `idx_ticket_arrival_lookup` covers `(arrival_time_local, arrival_day_offset)`.

Task 3.3 - Advanced Requirements (10 %)

Test environment. The import was tested on Windows 11 64-bit with Python 3.10.13 and on WSL2/Linux with Python 3.10.12. Both systems used the same source code and connected to the PostgreSQL database through `psycopg2`. The WSL2 environment was:

```
yukino@LAPTOP-JMOAUST0:/mnt/c/Users/Yiming Ma/Desktop/Claude_code_projects/DB_project_1/DB_project_1_by_copilot$ uname -a
Linux LAPTOP-JMOAUST0 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun  5 18:30:46 UTC 2025 x86_64 x86_64
x86_64 GNU/Linux
```

The CSV input sizes were:

CSV file	Rows
<code>airline.csv</code>	88
<code>airport.csv</code>	204
<code>passenger.csv</code>	1000
<code>region.csv</code>	261
<code>tickets.csv</code>	108820

Procedure. A clean database named `db_project_1` was created in PostgreSQL. Then the import was executed from PowerShell:

```
python import_data.py --host localhost --port 5432 --user postgres --password
<password> --database db_project_1
```

The script first executes `schema.sql`, then imports the source tables in foreign-key dependency order: `region`, `city`, `airport`, `airline`, `passenger`, `flight`, and `ticket_inventory`. During import, one airport row with an invalid IATA code was skipped as dirty data.

Result. The successful import produced the following final table sizes:

```
yukino@LAPTOP-JMOAUST0:/mnt/c/Users/Yiming Ma/Desktop/Claude_code_projects/DB_project_1/DB_project_1_by_copilot$ python3
import_data.py --host localhost --port 5432 --user postgres --password 123456 --database db_project_1
Skipped 1 airport row(s) with invalid IATA code
Import completed. Row counts:
region: 241
city: 196
airport: 206
airline: 88
passenger: 1000
flight: 21764
ticket_inventory: 108820
```

Efficiency design. The import uses `psycopg2.extras.execute_values` for batch inserts instead of inserting rows one by one. This reduces database round trips, which is especially important for the 108820 ticket rows. The script also deduplicates ticket inventory by `(flight_id, flight_date)` before bulk upsert, avoiding repeated conflict updates inside the same batch. The ticket table is split into `flight` and `ticket_inventory`, so repeated route/schedule information is stored once in `flight`, while date-specific price and remaining-seat information is stored in `ticket_inventory`.

Cross-system and measured time cost. The import time was measured on Windows from PowerShell and on WSL2/Linux using the shell `time` command.

```
Measure-Command { python import_data.py --host localhost --port 5432 --user postgres --password <password> --database db_project_1 }
```

```
(base) PS C:\Users\Yiming Ma\Desktop\Claude_code_projects\DB_project_1\DB_project_1_by_copilot> Measure-Command { python import_data.py --host localhost --port 5432 --user postgres --password 123456 --database db_project_1 }

Days           : 0
Hours          : 0
Minutes       : 0
Seconds       : 9
Milliseconds  : 286
Ticks         : 92861554
TotalDays     : 0.000107478650462963
TotalHours    : 0.00257948761111111
TotalMinutes  : 0.154769256666667
TotalSeconds  : 9.2861554
TotalMilliseconds : 9286.1554
```

```
time python3 import_data.py --host localhost --port 5432 --user postgres --password <password> --database db_project_1
```

```
yukino@LAPTOP-JMOAUST0:/mnt/c/Users/Yiming Ma/Desktop/Claude_code_projects/DB_project_1/DB_project_1_by_copilot$ time python3 import_data.py --host localhost --port 5432 --user postgres --password 123456 --database db_project_1
Skipped 1 airport row(s) with invalid IATA code
Import completed. Row counts:
  region: 241
  city: 196
  airport: 206
  airline: 88
  passenger: 1000
  flight: 21764
  ticket_inventory: 108820

real    0m5.548s
user    0m3.465s
sys     0m0.163s
```

System	Python	Runtime	Result
Windows 11 64-bit	Python 3.10.13	9.2861554 seconds	Success
WSL2/Linux	Python 3.10.12	5.548 seconds	Success

The same import script ran successfully on both systems without source-code changes.

Dirty data handling

- Arrival times encoded as `HH:MM(+1)` are split into a time value and an integer offset field, satisfying 1NF.

- Duplicate region names with different encodings are unified via the `REGION_ALIAS` dictionary.
- Airport IATA codes with length $\neq 3$ are discarded.
- Seats remaining are taken as the maximum value observed across all ticket rows for a given flight, ensuring capacity is not under-estimated.

Task 4: CRUD Operations via Program Access (30 %)

Overview

The CRUD operations are implemented in `cli.py`. The client talks to the FastAPI backend under `server/app`, while the backend handles validation, authorization, database transactions, and seat-count updates.

```
cd server
python -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
python ../cli.py
```

If no subcommand is provided, the CLI starts an interactive menu for login, ticket search, booking, order query, cancellation, logout, and administrator ticket generation. The current login state is stored in `.cli_session.json` and sent through the `X-Passenger-Id` request header.

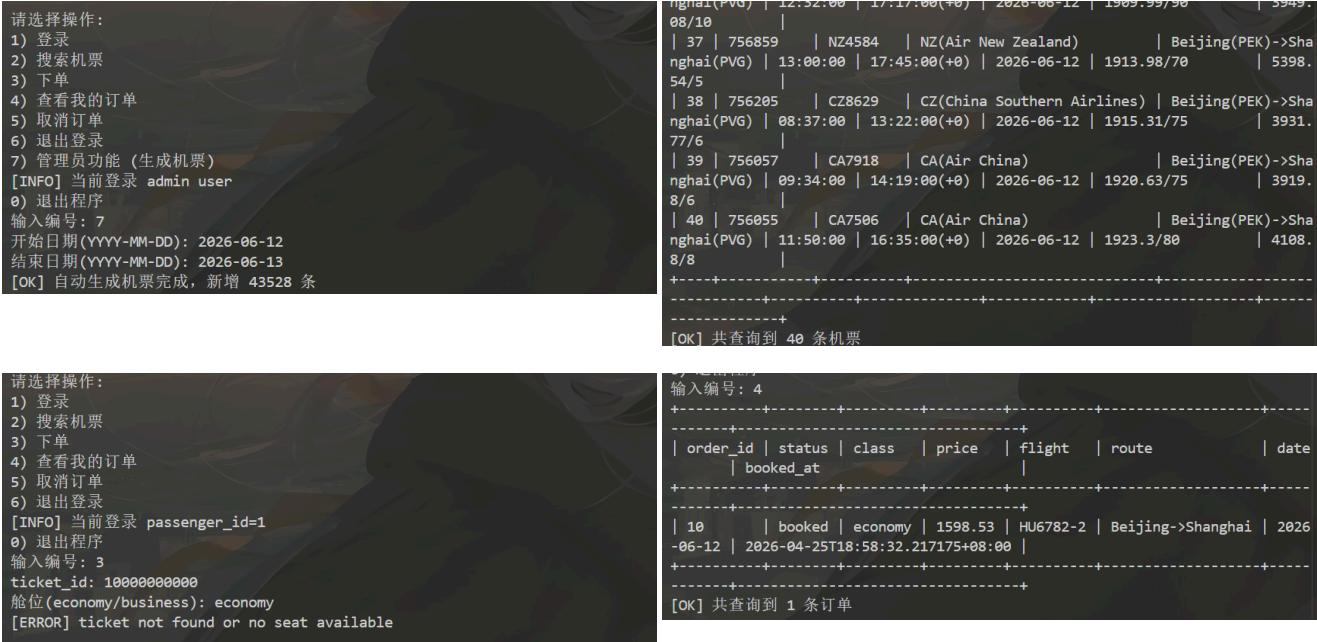
4.1 Basic CRUD Requirements

The CLI supports both direct subcommands and the interactive menu. The main commands used to verify the required operations are:

```
python cli.py login --mobile-number checker --password 114514
python cli.py generate --start-date 2026-04-10 --end-date 2026-04-16
python cli.py search --departure-city Rome --arrival-city London --date 2026-04-10
python cli.py book --ticket-id 100 --cabin-class economy
python cli.py orders
python cli.py cancel --order-id 10
```

Requirement	Endpoint and backend behavior
Generate ticket inventory	Admin-only call to <code>POST /api/v1/tickets/generate</code> ; missing (<code>flight_id</code> , <code>flight_date</code>) rows are inserted and duplicates are skipped.
Search tickets	Call to <code>/api/v1/tickets/search</code> with required city/date filters and optional airline/departure/arrival-time filters; results are printed as a table.
Book a ticket	Passenger-only call to <code>POST /api/v1/orders/book</code> ; the backend checks login, atomically decrements the selected cabin inventory, and inserts a <code>ticket_order</code> row.
List orders	Call to <code>/api/v1/orders/{passenger_id}</code> ; both booked and cancelled orders are shown with flight, route, cabin, price, date, and status.

Requirement	Endpoint and backend behavior
Cancel an order	Call to <code>/api/v1/orders/{passenger_id}/{order_id}/cancel</code> ; the order is locked, marked as <code>cancelled</code> , and the corresponding seat count is restored.

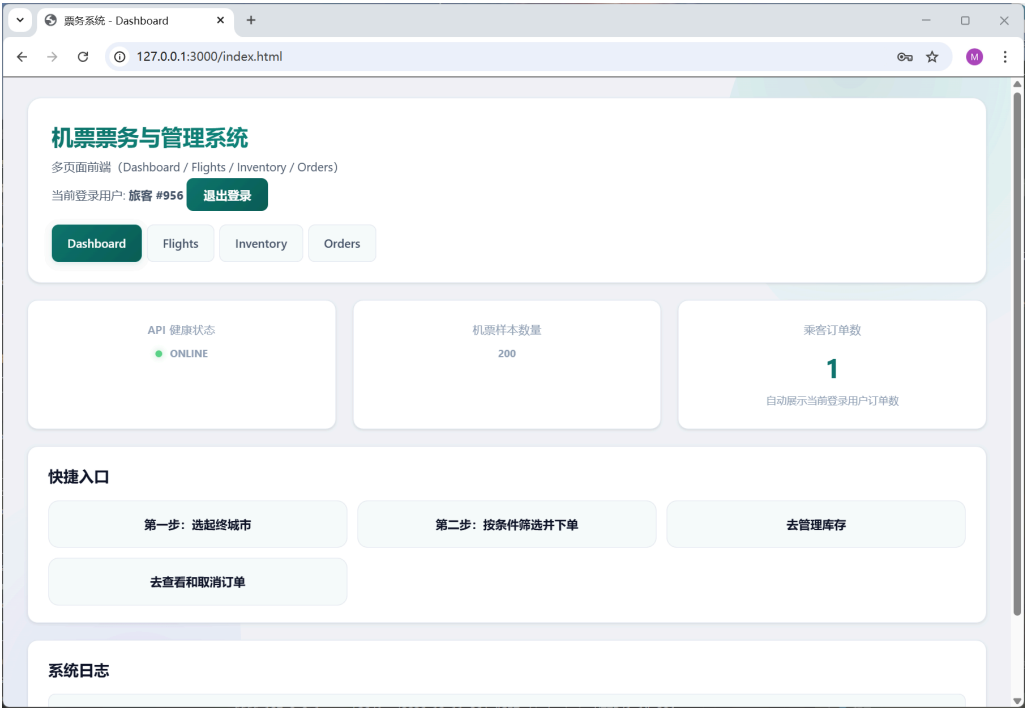


4.2 Login, Logout, and Authorization

`login` calls `/api/v1/auth/login`; a successful login writes `.cli_session.json`, and `logout` removes it. For protected order APIs, `orders.py` checks that the requested passenger id matches the logged-in id in `X-Passenger-Id`. Ticket generation is further restricted to the administrator account `checker / 114514`.

4.3 Advanced and Bonus Features

Feature	Implementation
Interactive CLI	Running <code>python cli.py</code> opens a menu for login, search, booking, order query, cancellation, logout, and admin generation.
Session workflow	<code>.cli_session.json</code> keeps the logged-in identity so repeated operations do not require retyping <code>--passenger-id</code> .
Atomic booking	<code>OrderRepository.decrement_seat_and_get_price()</code> updates inventory only when seats are available, then creates the order.
Soft cancellation	Cancellation changes <code>status</code> to <code>cancelled</code> and restores the selected cabin inventory.
Efficient generation	Ticket generation uses PostgreSQL set-based SQL: <code>generate_series</code> , <code>CROSS JOIN</code> , CTE filtering, <code>INSERT ... SELECT</code> , and <code>ON CONFLICT DO NOTHING</code> .
Permission management	Passengers can view/cancel only their own orders, and ticket generation requires administrator identity.
GUI bonus	The <code>client/</code> frontend provides login, dashboard, flight query, ticket selection, inventory, and order-management pages using the same backend APIs.



The contact-management bonus item was **not implemented**. The current booking flow only supports creating orders for the logged-in passenger, so no separate contact table was added.

Attachments

The following files are submitted alongside this report:

```
DB_project_1/
├── ER_diagram.png
├── ER-diagram by DataGrip.png
├── schema.sql
├── import_data.py
├── task3_accuracy_queries.sql
├── cli.py
├── server/
├── client/
└── Archive/
```